



Università
Ca'Foscari
Venezia

Blind Search

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

Andrea Torsello

Dip. Sc. Ambientali, Informatica, Statistica
Università Ca'Foscari Venezia

Search

Search plays a key role in many parts of AI. These algorithms provide the conceptual backbone of almost every approach to the systematic exploration of alternatives.

One general approach to problem solving in AI is to reduce the problem to be solved to one of searching a graph.

To use this approach, we must specify what are **the states**, the **actions** and the **goal test**.

Our goal is to plan a series of action that takes us from an initial state to a goal state.

Search

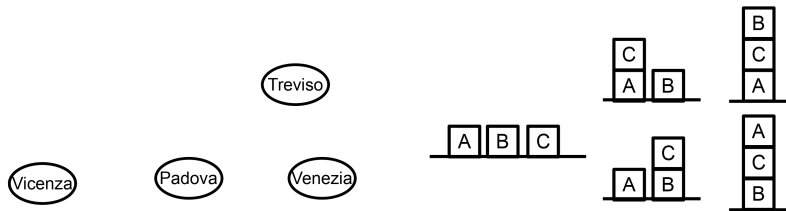
The search can be:

- ▶ **Uninformed (or blind):** I know nothing more than **the neighbors** of the current state (states reachable through a single action)
- ▶ **Informed (or heuristic):** I have a means to assess progress toward the goal

Graph-Based Search: States

What are the states? (all relevant aspects of the problem)

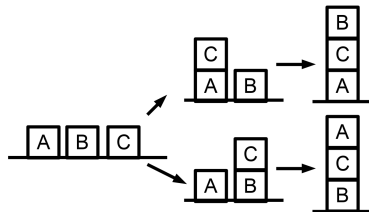
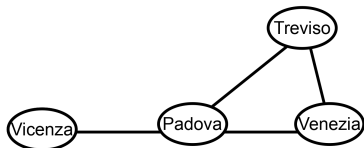
- ▶ Arrangement of parts (to plan an assembly)
- ▶ Position of Trucks (to plan package distribution)
- ▶ Cities (to plan a trip)
- ▶ Set of facts (e.g. to prove theorems)



Graph-Based Search: Actions

What are the actions (operations)? (neighboring states)

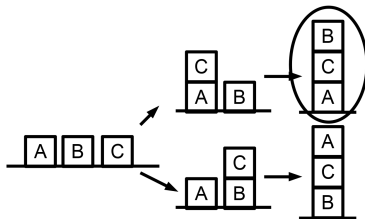
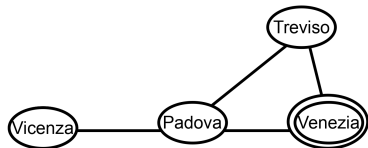
- ▶ Assemble two parts
- ▶ Move a truck to a new position
- ▶ Drive to a new city
- ▶ Apply a theorem to derive new fact



Graph-Based Search: Goal

What is the goal test? (Condition for success)

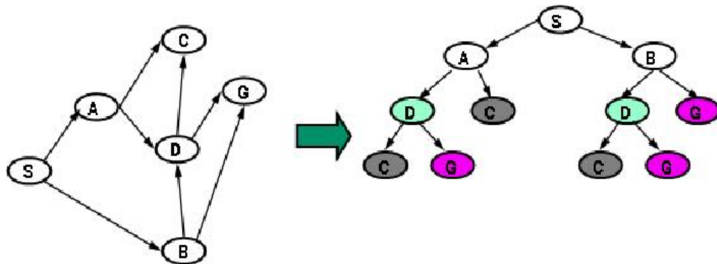
- ▶ All parts in place
- ▶ All package delivered
- ▶ Destination reached
- ▶ Derived goal fact



Graph Search as Tree Search

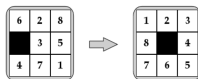
We can turn graph search problems into tree search problems by

- ▶ Replacing undirected links by 2 directed links (back and forward)
- ▶ Avoiding loops in path (or keeping track of visited nodes)

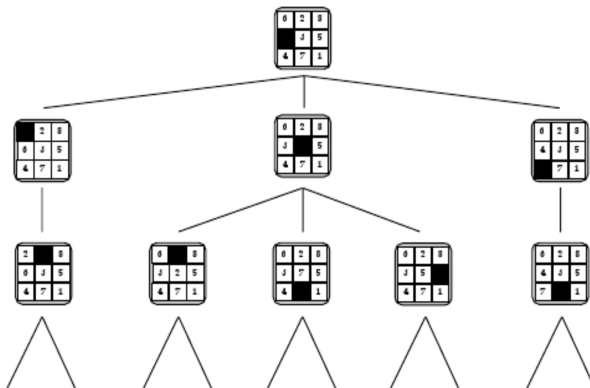


Example: The Game of 8

Goal: move the squares from initial to final configuration



The search tree:



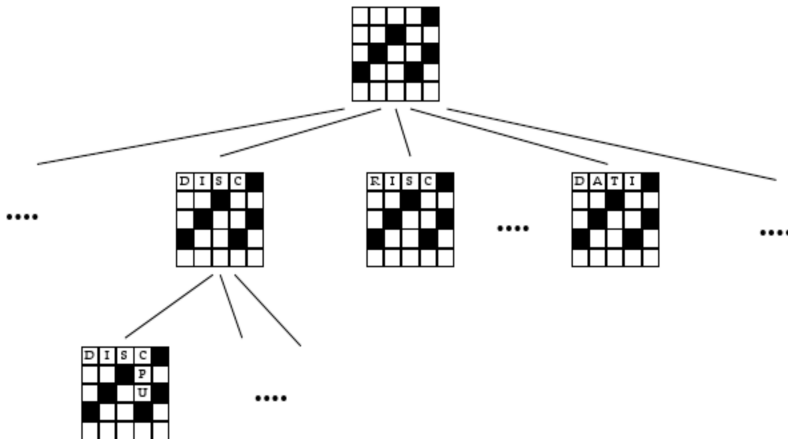
Example: Crosswords

There are 26^{19} possible ways to fill the boxes of the crosswords puzzle on the right, but only a tiny fraction is legal, *i.e.*, forms real words

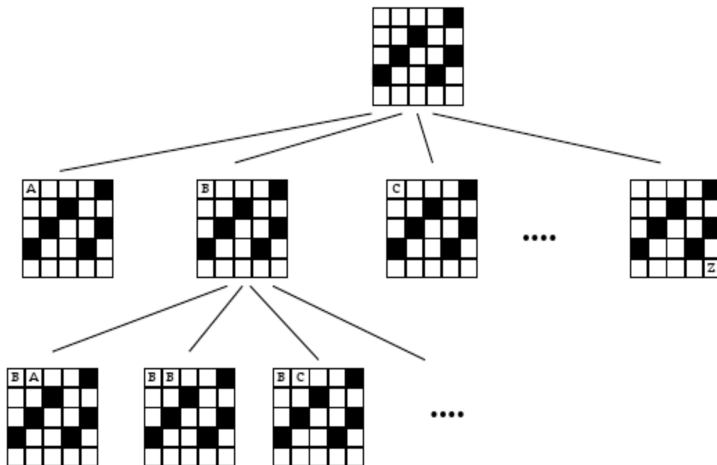
R	I	S	C	
O	F		D	O
M		C	R	
	T	P		P
M	O	U	S	E

There are several ways in which you can construct the search space, depending on your definition of states and actions

Crosswords: State Space 1



Crosswords: State Space 2



Evaluation

There are 4 criteria to evaluate a search algorithm:

Completeness: Will the algorithm find a solution if one exists?

Optimality: Will the algorithm find the “best” solution when more than one exist?

Time/**Temporal Complexity:** How long will it take to find a solution?

Space/**Spatial Complexity:** How much memory will it require?

Stab of Search Procedure

Example (Stab of search procedure)

```
search(G,source,is_goal):  
    L=[source] # list of visited but  
                # not expanded nodes  
    while L:  
        n = extract_from(L) #expand node n  
        if is_goal(n): # goal node found  
            return path(source,n)  
  
        # goal not found, visit adjacent nodes  
        children = G.adjacent(n)  
        insert_into(L, children)  
    return FAIL # goal not present in G
```

Stab of Search Procedure

Nodes are:

visited when they are encountered and placed in the list L

expanded when they are removed from L, subjected to the goal test, and its neighboring nodes are inserted into L (visited)

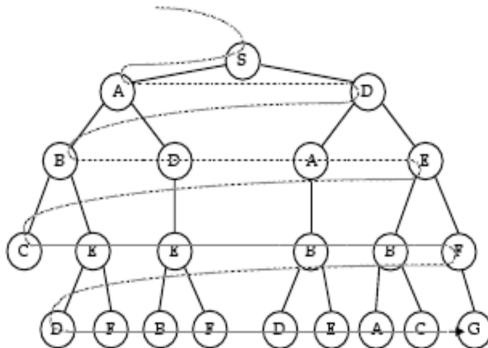
The type of search depend on how you `extract_from` and `insert_into` L

Breadth-First Search

Search is performed by extracting from the front and inserting on the back of the list (FIFO)

The algorithm proceed level by level, hence it is

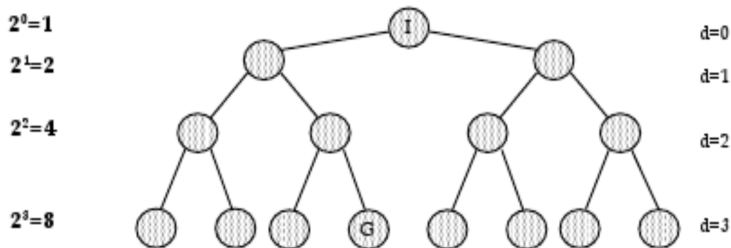
- ▶ Complete
- ▶ Optimal (will find the solution at smallest depth)



Breadth-First: Temporal Complexity

Simplifying hypotheses:

- ▶ The search tree has constant branching factor b
- ▶ The first goal is at depth d



Observation: the number of nodes at level d is b^d

Breadth-First: Temporal Complexity

The number of nodes to examine to reach depth d is:

$$1 + b + b^2 + \dots + b^{d-1} = \frac{b^d - 1}{b - 1}$$

The number of nodes to examine to reach depth d is:

If $d=3$, then we will get 4.5,
but it is searching at level d ,
not $d-1$

$$\frac{1 + b^d}{2}$$

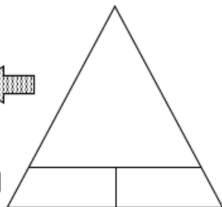
This number represents a midpoint
in the search effort but does not
correspond directly to a specific
depth level.

Search up to
level $d-1$



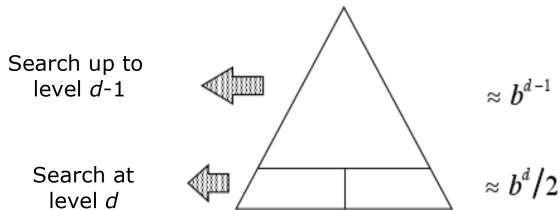
$$\approx b^{d-1}$$

Search at
level d



$$\approx b^d/2$$

Breadth-First: Temporal Complexity



Time complexity is dominated by search at the last level!

Adding the two terms we obtain:

$$\frac{b^{d+1} + b^d + b - 3}{2(b-1)} = O(b^d)$$



Breadth-First: Temporal Complexity

BFS explores all nodes level by level up to the solution depth d . If the solution (goal node) is at level d , BFS will have examined all nodes from the root to that level, which amounts to approximately $O(b^d)$ nodes.

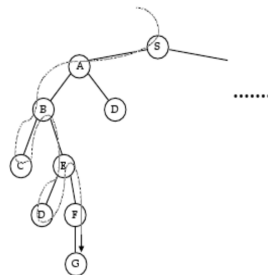
Before the algorithm examines the first node at depth k , it must have memorized all of them (b^k)

The memory required to reach the goal node at level d is $O(b^d)$

Depth-First Search

Search is performed by extracting and inserting from the front of the list (**LIFO**)

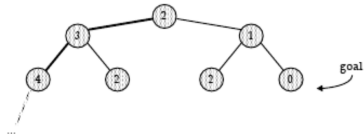
Results in a search process that goes down on one side of the search tree before coming back up to check other paths



Depth-First Search

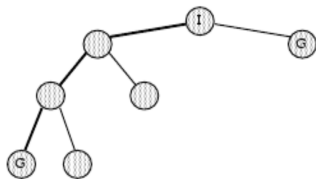
The algorithm is not complete:

If the tree has infinite depth, the algorithm might get stuck in an infinite branch and not find a solution even when one exists



The algorithm is not optimal:

If the problem has more than one solution the algorithm is not guaranteed to find the one at minimum depth



Depth-First Search: Temporal Complexity

Simplifying hypotheses:

- ▶ The search tree has constant branching factor b
- ▶ The tree has depth d
- ▶ There is only one goal and it is at depth d

In the best case the goal is at the far left of the tree. Hence, the number of nodes to examine is:

$$d + 1$$

In the worst case it is at the far right:

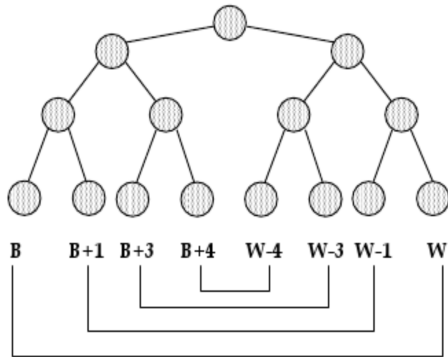
$$1 + \dots + b^d = \frac{b^{d+1} - 1}{b - 1}$$

Taking the average we have:

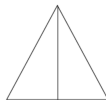
$$\frac{b^{d+1} + bd + b - d - 2}{2(b - 1)}$$

Depth-First Search: Temporal Complexity

Why is it reasonable to take the average?



For large d , the average search time is around $b^d/2 = O(b^d)$, i.e., the time to visit half the tree

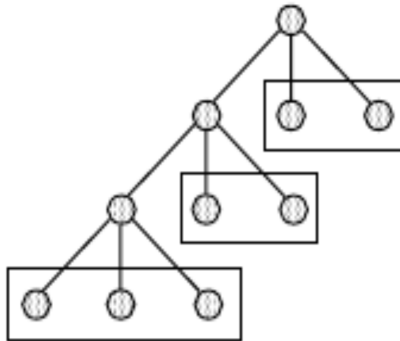


$$\approx b^d/2$$

Depth-First Search: Spatial Complexity

The maximum memory required is

$$d(b - 1) + 1 = O(db)$$



Iterative Deepening

Basic idea:

- ▶ perform depth-first search up-to depth C
- ▶ Increase maximum depth C until goal is found

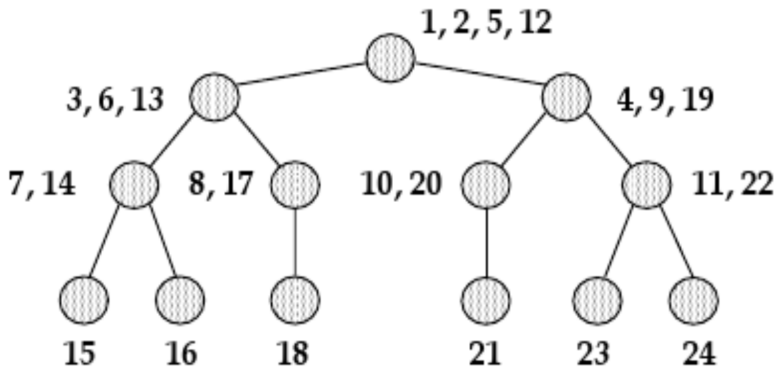
Iterative Deepening

Example (Iterative Deepening)

```
iterative_deepening(G,source,is_goal):  
    C=0  
    while C<max_depth:  
        L=[source] # reinitialize list and  
                  # restart with new C  
        while L:  
            n = L.pop_left()  
            if is_goal(n):  
                return path(source,n)  
            # visit adjacent nodes if depth<C  
            if depth(n)<C:  
                children = G.adjacent(n)  
                L.append(children)  
        C+=1 # inrease C and restart  
    return FAIL
```

Iterative Deepening

The algorithm proceeds level by level, thus, just like breadth first, it is complete and optimal



Here the numbers represent the expansion order of the nodes

Iterative Deepening: Temporal Complexity

In the last iteration (at level d), the average nodes examined is:

$$\frac{b^{d+1} + db + b - d - 2}{2(b - 1)}$$

In each previous iteration the number of nodes examined is:

$$\frac{b^{j+1} - 1}{b - 1}$$

Hence, in total, from all the previous iterations, we have:

$$\sum_{j=1}^{d-1} \frac{b^{j+1} - 1}{b - 1} = \frac{b^{d+1} - db - b + d}{(b - 1)^2}$$

Adding we have

$$\frac{b^{d+2} + b^{d+1} + (d + 1)b^2 - (4d - 5)b + 3d + 2}{2(b - 1)^2} = O(b^d)$$



Iterative Deepening: Spatial Complexity

Since at each iteration the algorithm performs a depth-first search, the memory requirement is the same as Depth-First:

$$d(b - 1) + 1 = O(db)$$

Uniform Cost Search

Until now we have evaluated optimality only in terms of number of steps (actions)

Uniform Cost is a blind search algorithm that is optimal according to any specified path length function

Can be obtained from the generic search stab function by selecting from L the node with the **minimal path length** from the root

Can be implemented with a heap or any efficient priority queue implementation

Uniform Cost: Time/Space Complexity

Uniform cost is guided by path length rather than depth, so its time complexity cannot easily be characterized in terms of b and d

Rather, let C^* be the path length of the optimal solution and ϵ the minimum cost of any action. Then the worst-case time and space complexity is

$$O(b^{1+\lfloor C^*/\epsilon \rfloor})$$

which can be much greater than b^d

Uniform cost can explore large trees of small steps before exploring paths with large and potentially useful steps.

When all steps costs are equal, uniform cost is similar to breadth first.